BlueJay

Web Point of Sale System

Team Structure

Sara Orion (Team & Security Lead):

- Project governance, milestone compliance, and security standards.

Kimleng Lim (Design & Implementation Lead):

- UI architecture, UX flows, and core feature delivery.

Krizma Nagi (Requirements Lead):

- Backlog grooming, user story definitions, and acceptance criteria.

Andy Chang (Configuration Lead):

- AWS cloud infrastructure, CI/CD pipelines, and environment management.

Italia Tran (QA Lead):

- Test strategy, automated tracking, and pull request validation rules.

Project Vision

A web point of sale system for small grocery and retail stores

THE PROBLEM

Small grocery and retail stores rely on legacy terminal software, paper stock tracking, and manual invoice entry. Existing POS systems are expensive or desktop-only.

WHO IT'S FOR

Store administrators, stock managers, and cashiers in small grocery and retail stores.

WHAT BLUEJAY DOES

Turns a legacy console POS into a secure, multi-user web app for checkout, inventory, and reporting, with role-based access.

Requirements

Requirements workflow

SPPP requirements

24 features in 4 priority tiers, plus nonfunctional

↓ audited and converted

GitHub Projects backlog

Every feature as a user story, grouped under epics

↓ written and estimated

Testable user stories

Given/When/Then criteria, tasks, points, owner

↓ system tested

Done

A story is accepted once its system test passes

Delivery

Backlog management

Audit	Matched the board to the SPPP, filled gaps, removed 15 duplicate stories
Estimation	Story points for every story with planning poker
Scope	Core POS before AI, low-stock alerts moved to managers, reports combined

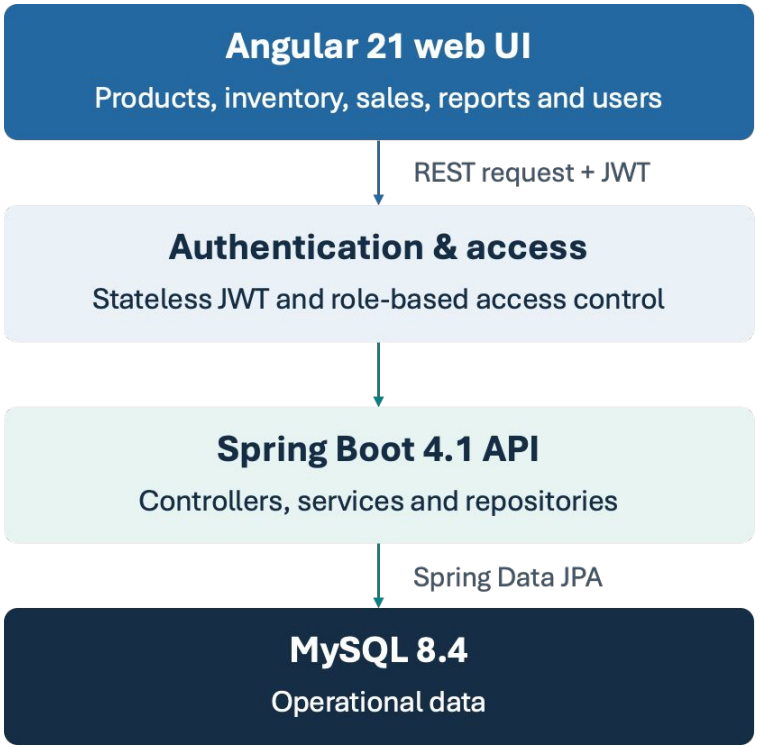
Implemented Features

#18	Cashier session lockout and idle timeout
#57	Core inventory report
#29	Sales report by date range
#30	Sales by product analysis

System Design

Angular Frontend + Spring Boot Backend

Application workflow



Frontend + Backend

Technology stack

Angular 21 + Java 21 / Spring Boot 4.1
Reusable UI components and clean feature slices

Grocery operations

Products	Catalog, categories and barcode lookup
Inventory	Stock entry, health and shortage visibility
Sales	POS checkout
Reports	Stock and cost data
Users	Admin directory and role-based creation

Data & developer tools

MySQL + JPA persistence, Flyway migrations
Scalar UI + OpenAPI documentation

Testing

- Two types of testing for each user story
  - Automated unit tests for frontend and backend
  - Manual system tests to validate and verify changes to the system
- For any methods that are created
  - Should be a minimum of two unit tests (one happy and one sad path)
- System tests needs two passes in order to be complete
  - First pass is done by the developer of the user story
  - Second pass is done by the QA lead
- Code coverage and reporting for testing includes
  - JaCoCo framework for backend
  - Angular framework for frontend

Unit Test Summary

Frontend Test Summary

Feature	Test Files	Passed	Failed	Duration	Statements	Branches	Functions	Lines
Application	4	15	0	0.5s	64.5%	76.7%	37.9%	64.5%
Auth	4	26	0	0.6s	81.9%	90.0%	76.9%	81.9%
Dashboard	1	2	0	0.0s	100.0%	100.0%	100.0%	100.0%
Inventory	5	24	0	0.7s	86.8%	98.3%	80.0%	86.8%
Products	4	43	0	5.3s	94.2%	96.0%	83.3%	94.2%
Reports	6	29	0	1.5s	92.0%	97.7%	75.6%	92.0%
Sales	2	18	0	1.7s	90.2%	95.7%	75.0%	90.2%
Users	5	18	0	0.8s	81.5%	92.0%	69.5%	81.5%

Overall: 31 test files · 175 passed · 0 failed · 11.2s

Backend test summary

Category	Test Classes	Passed	Failed	Skipped	Duration	Instructions	Lines	Branches
Auth	17	33	0	0	86.4s	65.0%	65.4%	29.3%
Inventory	3	13	0	0	0.8s	98.1%	96.3%	71.4%
Product	9	36	0	0	2.8s	93.4%	94.9%	83.3%
Report	6	31	0	0	2.2s	72.6%	74.3%	50.0%
Sales	3	8	0	0	1.8s	79.2%	77.5%	77.3%

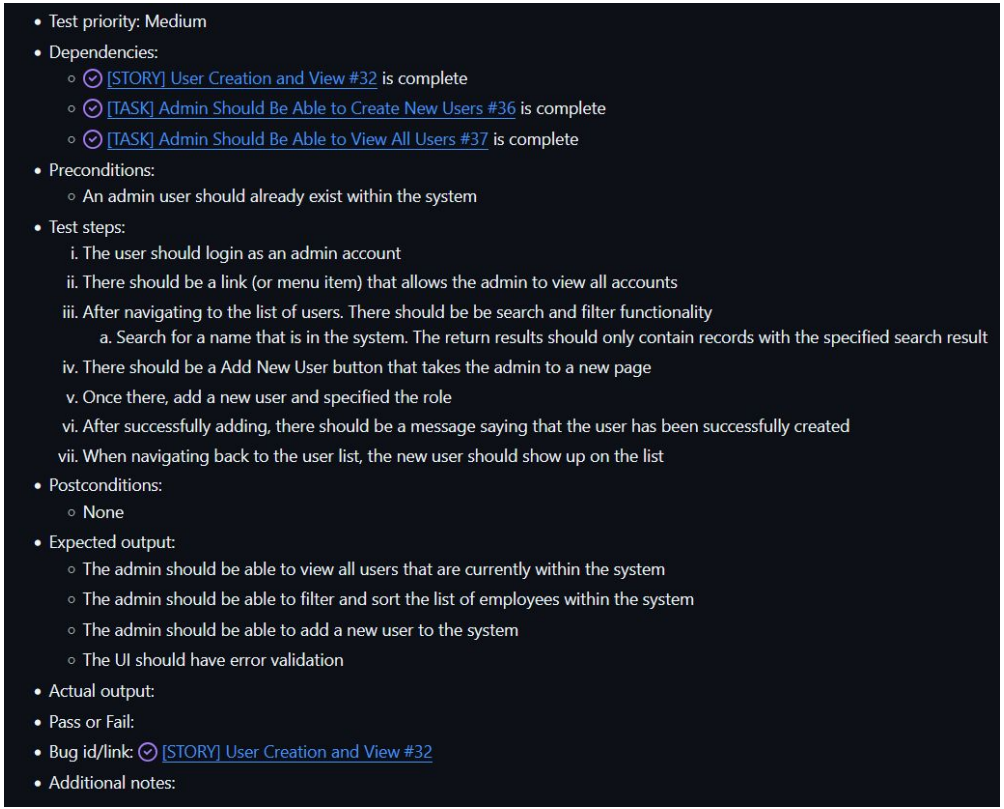
Total

- **Tests:** 121 total · 121 passed · 0 failed · 0 errors · 0 skipped
- **Duration:** 94.0s
- **Coverage:** 78.8% instructions, 78.3% lines, 52.9% branches

Coverage totals

Metric	Covered	Missed	Coverage
Instructions	2780	746	78.8%
Lines	624	173	78.3%
Branches	91	81	52.9%

System Testing



- Each user story needs to have its own system test task
- The system test is completed when the test steps are completed for both passes
- 17 User Stories completed
  - 9 in Iteration 2
  - 8 in Iteration 3

Deployment & CI/CD

GitHub Actions tests every change and deploys develop and main to AWS EC2

CI/CD pipeline

Push or pull request

Every change to develop or main

↓ must pass to merge

Automated tests

Backend: JUnit + JaCoCo Frontend: Vitest + coverage

↓ merge to develop or main

Deploy over SSH

GitHub Actions logs in to EC2 with repository secrets

↓ prune old images, redeploy

Docker Compose on AWS EC2

MySQL 8.4, Spring Boot API and Angular web containers

Workflows & infrastructure

Workflows

Backend	maven-test.yml: compile, run JUnit tests, post test and coverage summary
Frontend	frontend-test.yml: install, run Vitest with coverage, post summary
Deploy	deploy.yml: on push to develop or main, prune images, run redeploy.sh

Hosting & secrets

Hosting	AWS EC2 runs the app and the Scalar API docs
Secrets	SSH key in GitHub secrets; DB password and JWT key in AWS Parameter Store

Lesson learned

Deploys failed when the server disk filled up, so every deploy now prunes old Docker images first

Security

Stateless Authentication & JWT Token Handling

- Spring Security and JJWT together created a stateless session management.
- Validated **Bearer** tokens containing user roles on every incoming request.

Stateful Token Revocation & Logout Architecture

- In-memory **TokenBlacklistService** revokes tokens upon logout.
- Automatically purges expired entries while keeping API endpoints secure.

REST API Protection & CORS Configuration

- Secured API restricting CORS origins to trusted clients
- Disabled CSRF for stateless requests, and
- Returned 401/403 status codes instead of HTML redirects.

Role-Based Access Control (RBAC) & Secure User Loading

- BCrypt password hashing
- Java 21 **CustomUserDetails** records map database permissions.
- Restrict API access via granular method security.

AWS Parameter Store Integration

- Secured sensitive application credentials as encrypted **SecureString** with KMS decryption and IAM access.
    - Database passwords
    - JWT signing key

AI Usage

Project-wide summary from the team AI log and the AI-usage headers in our code

How we used AI

Draft
First drafts of code, tests, user stories and documents

Review
Checked against our design: 31 of 54 logged uses modified, 1 rejected

Verify
CI unit tests, code review and system tests

Disclose
AI-usage header in each file and a shared team AI log

Project-wide metrics

AI contribution

Code	About 86% AI-assisted, from headers in 186 files
Tests	About 87% AI-assisted (50 test files)
Documents	About 72% across 10 logged SPPP, SDD and STD sections

Tools (54 log entries)

Most used	Gemini (23), Claude (13), Copilot (12)
Also used	DeepSeek (3), Google AI (2), ChatGPT (1)

Agile Process & Iteration Evolution

GitHub Projects Architecture:

- Tracked work using Kanban boards categorized by *Epic, User Story, Task,* and *Test*.

Integrated Test Tracking:

- Every User Story included dedicated implementation subtasks paired with mandatory System Test subtasks.

Velocity Calibration:

- Sprints initially suffered from over-estimation and task rollover.

Agile Realignment:

- Scaled back non-essential scope in later sprints to guarantee 100% completion of core functionality.

Risk Management, Achievements & Lessons Learned

Infrastructure Risk & Mitigation:

- Resolved an AWS storage outage right before the Iteration 2 deadline under tight submission constraints.

Major Achievement:

- Successfully delivered a stable, fully functional core application through strong late-stage team coordination.

Key Lessons Learned:

- Calibrate sprint scope based on historical velocity rather than ideal capacity.
- Actively monitor cloud environment storage and resource quotas to prevent release-day bottlenecks.